

**INSTITUTO
FEDERAL**
Santa Catarina

Orientação a Objetos em JavaScript (parte 2)

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina
Câmpus São José

PROGRAMAÇÃO
FRONTEND II

Análise e Desenvolvimento
de Sistemas

cores

#111027

#277756

#16ABCD

#FFF4EC

#C74E23

fontes

Fira Sans Extra Condensed

Ubuntu

Roboto Mono



Subclasses

subclasses

- herdam as propriedades e os métodos das classes (superclasses)
- podem definir seus próprios métodos
- podem sobrescrever os métodos das superclasses

Subclasses

criando subclasses

- utilizando protótipos (herança simulada)

exemplos

```
function Retangulo(largura, altura) {  
  this.largura = largura;  
  this.altura = altura;  
}  
  
Retangulo.prototype = {  
  area: function() {  
    return this.largura * this.altura;  
  },  
  perimetro: function() {  
    return 2*(this.largura + this.altura);  
  }  
}
```

Subclasses

criando subclasses

- utilizando protótipos (herança simulada)

exemplos

```
function Retangulo(largura, altura) {  
  this.largura = largura;  
  this.altura = altura;  
}
```

```
function Quadrado(lado) {  
  Retangulo.call(this, lado, lado);  
}
```

```
Quadrado.prototype = Object.create(Retangulo.prototype);
```



Subclasses

criando subclasses

- utilizando protótipos (herança simulada)
- **definidas com extends**
 - o construtor deve invocar o construtor da superclasse (super)
 - this só pode ser invocado depois de super (a subclasse é iniciada depois da superclasse)

exemplos

```
class Retangulo {
    constructor(largura, altura) {
        this.largura = largura;
        this.altura = altura;
    }
    area() {
        return this.largura * this.altura;
    }
    perimetro() {
        return 2*(this.largura + this.altura);
    }
}

class Quadrado extends Retangulo {
    constructor(lado) {
        super(lado, lado);
    }
}
```

Subclasses

criando subclasses

- utilizando protótipos (herança simulada)
- definidas com extends
- **delegação (composição)**
 - não é herança!
 - delegação de comportamento para uma classe instanciada por outra

exemplos

```
class Quadrado {  
  constructor(lado) {  
    this.retangulo = new Retangulo(lado,  
    lado);  
  }  
  area() {  
    return this.retangulo.area();  
  }  
  perimetro() {  
    return this.retangulo.perimetro();  
  }  
}
```



Classes Abstratas

o que são

- classes que não podem ser instanciadas diretamente
- podem conter métodos concretos ou abstratos
- os métodos abstratos devem ser implementados pelas subclasses
- os métodos concretos podem ser reutilizados ou sobrescritos pelas subclasses
- não existem formalmente em JS!

exemplos

```
class Figura {  
  area() {  
    throw new Error('Método não  
      implementado');  
  }  
  perimetro() {  
    throw new Error('Método não  
      implementado');  
  }  
}
```



Classes Abstratas

o que são

- classes que não podem ser instanciadas diretamente
- podem conter métodos concretos ou abstratos
- os métodos abstratos devem ser implementados pelas subclasses
- os métodos concretos podem ser reutilizados ou sobrescritos pelas subclasses
- não existem formalmente em JS!

exemplos

```
class Retangulo extends Figura {  
  constructor(largura, altura) {  
    super();  
    this.largura = largura;  
    this.altura = altura;  
  }  
  area() {  
    return this.largura * this.altura;  
  }  
  perimetro() {  
    return 2 * (this.largura +  
      this.altura);  
  }  
}
```

Objetos

o que são

- coleções não ordenadas de propriedades, cada uma com um nome e um valor
- propriedades podem ser adicionadas e excluídas
- nomes geralmente são strings
- valores podem ser primitivos ou objetos
- **em JS, funções são objetos!**

exemplos

```
let vazio = {};
```

```
let livro = {  
  titulo: "JavaScript",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "David",  
    sobrenome: "Flanagan",  
  },  
};
```



Objetos

o que são

- coleções não ordenadas de propriedades, cada uma com um nome e um valor
- propriedades podem ser adicionadas e excluídas
- nomes geralmente são strings
- valores podem ser primitivos ou objetos
- **em JS, funções são objetos!**

exemplos

```
let retangulo = {  
  
  largura: 2,  
  altura: 5,  
  
  area: function () {  
    return this.largura * this.altura;  
  },  
  perimetro: function () {  
    return 2 * (this.largura + this.altura);  
  }  
}  
  
retangulo.area(); // 10
```



Objetos

o que são

- coleções não ordenadas de propriedades, cada uma com um nome e um valor
- propriedades podem ser adicionadas e excluídas
- nomes geralmente são strings
- valores podem ser primitivos ou objetos
- **em JS, funções são objetos!**

exemplos

```
let retangulo = {  
  
  largura: 2,  
  altura: 5,  
  
  area() {  
    return this.largura * this.altura;  
  },  
  perimetro() {  
    return 2 * (this.largura + this.altura);  
  }  
}  
  
retangulo.area(); // 10
```



Métodos

métodos estáticos

- associados diretamente a uma classe e não às suas instâncias
- não estão disponíveis às instâncias, apenas à classe
- definidos com `static` antes do nome do método
- usos comuns
 - métodos utilitários
 - operações globais

exemplos

```
class Retangulo {  
  
    constructor(largura, altura) {  
        this.largura = largura;  
        this.altura = altura;  
    }  
  
    static area(largura, altura) {  
        return largura * altura;  
    }  
}  
  
Retangulo.area(2, 5); // 10
```

Propriedades

tipos de acesso

■ públicas

- acessadas diretamente

exemplos

```
class Retangulo {  
    constructor() {  
        this.largura = 2;  
        this.altura = 5;  
    }  
    area() {  
        return this.largura * this.altura;  
    }  
    perimetro() {  
        return 2 * (this.largura + this.altura);  
    }  
}
```

Propriedades

tipos de acesso

- públicas
 - acessadas diretamente

exemplos

```
class Retangulo {  
  
    largura = 2;  
    altura = 5;  
  
    area() {  
        return this.largura * this.altura;  
    }  
    perimetro() {  
        return 2 * (this.largura + this.altura);  
    }  
}
```



Propriedades

tipos de acesso

- públicas
 - acessadas diretamente
- privadas
 - acessadas apenas internamente

exemplos

```
class Retangulo {  
  
    #largura = 2;  
    #altura = 5;  
  
    area() {  
        return this.#largura * this.#altura;  
    }  
    perimetro() {  
        return 2 * (this.#largura + this.#altura);  
    }  
}
```

Propriedades

tipos de acesso

- públicas
 - acessadas diretamente
- privadas
 - acessadas apenas internamente
 - têm que ser declaradas antes de usadas

exemplos

```
class Retangulo {  
  
    #largura;  
    #altura;  
  
    constructor(largura, altura) {  
        this.#largura = largura;  
        this.#altura = altura;  
    }  
  
    area() {  
        return this.#largura * this.#altura;  
    }  
    perimetro() {  
        return 2 * (this.#largura + this.#altura);  
    }  
}
```

Propriedades

tipos de acesso

- públicas
 - acessadas diretamente
- privadas
 - acessadas apenas internamente
 - têm que ser declaradas antes de usadas
- **getters e setters**
 - permitem acessar e modificar propriedades de forma controlada
 - propriedades parecem ser acessadas diretamente
 - permitem o **encapsulamento** do estado interno

exemplos

```
class Retangulo {
  #largura;
  #altura;
  constructor(largura, altura) {
    this.#largura = largura;
    this.#altura = altura;
  }
  get largura() {
    return `${this.#largura} cm`;
  }
  set largura(valor) {
    if (typeof valor === 'string') {
      valor = parseInt(valor);
    } else if (typeof valor !== 'number') {
      throw new Error('A largura deve ser um número');
    }
    this.#largura = valor;
  }
}
```

Métodos

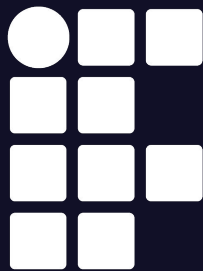
polimorfismo

- mesmo nome, diferentes assinaturas
- não nativo no JS
- simulado verificando os parâmetros dentro da função

exemplos

```
function calcularArea(...args) {  
  
    if (args.length === 1) {  
        return Math.PI * args[0] ** 2;  
    } else if (args.length === 2) {  
        return args[0] * args[1];  
    }  
}  
  
console.log(calcularArea(5));  
// Área do círculo: 78.54  
  
console.log(calcularArea(4, 6));  
// Área do retângulo: 24
```





**INSTITUTO
FEDERAL**
Santa Catarina

Orientação a Objetos em JavaScript (parte 2)

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br

